RepoGuide: Project Plan

Oct 31, 2026 · @Moizz

Vision

RepoGuide is a tutor for code: paste a GitHub URL and it teaches you the repository out loud, then answers your spoken questions with proof from the code.

One-line pitch: Paste any repo. Get a guided, spoken tour you can interrupt, grounded in code that was actually run.

End state: a person lands on an unfamiliar repo, gets a 3 to 5 minute tour that highlights files while it talks, interrupts with a voice question, and leaves with a concrete plan for a first contribution.

Three promises the product must keep:

- Every claim cites a file and line range, shown on screen as it is spoken.
- Explanations come from observed behavior (install, run, test) when the repo can be run, and from static analysis when it cannot. The screen says which.
- It remembers what you have already learned and does not repeat it.

RepoGuide is a working title; rename freely.

Problem and users

Reading an unfamiliar repository is slow, and today's tools fail in predictable ways.

The pain

- Onboarding to a new codebase takes days, and the README rarely explains the architecture.
- Chat-with-repo tools answer from retrieved snippets, so they miss how parts connect and sometimes name files that do not exist.
- Explanations arrive as a wall of text with no order, so beginners do not know where to start.
- Nothing bridges "I understand this" and "I can change this".

Target users, in priority order

1. Student and junior contributors picking up a first issue in a repo they did not write. This is the primary user, with a deadline and no map.
2. Hackathon teams inheriting, forking or judging unfamiliar code.
3. Learners studying well-known open-source projects.

Design rule: when a feature does not help the primary user get from "new repo" to "first correct change", it is cut.

How it differs from existing tools

Tools in the chat-with-repo category, DeepWiki being the best known, read code and write text answers. RepoGuide competes on three things they do not combine: observed behavior, a spoken guided tour, and a path to a first contribution. This comparison comes from general knowledge of the category, so audit each competitor's current features before pitching.

Dimension	Typical chat-with-repo tool	RepoGuide
Source of truth	Static code and docs	Static code plus observed test and run behavior
Output	Text answers or a generated wiki	Spoken guided tour synced to a code viewer
Interaction	Type a question	Interrupt by voice mid-tour
Proof	Sometimes cites files	Every claim cites file and line, validated, with an "observed" badge
End goal	Understanding	Understanding, then an issue-level contribution plan
Memory	Usually per chat	Learner profile and covered topics per repo

The three differentiators, by value

1. Execution-grounded explanations. Hardest to build, hardest to copy.
2. A guided, interruptible tour. This is the demo moment.
3. A contribution planner. This is the reason to come back.

Build order is the reverse of risk: tour first, planner second, execution last. Execution can fail on many repos, so it must never be required for the product to work.

User experience

The screen has three zones: a read-only code viewer on the left, a transcript with citations on the right, and a mic button plus text box along the bottom. Everything the voice says is also on screen.

Flow A: paste and map (first 60 seconds)

1. The user pastes a GitHub URL and picks a level (beginner, intermediate, expert) and a goal (understand, contribute, review).
2. A progress strip shows real stages: fetching, mapping, indexing, and running when execution is enabled.
3. Within about 30 seconds an overview card appears: what the project does, the stack, the entry points and how to run it. Deeper analysis keeps running behind it.

Flow B: guided tour

1. The user presses Start tour. The voice narrates 5 to 8 stops, each about 30 seconds.
2. At each stop the viewer opens the file and highlights the exact lines being discussed, moving the highlight sentence by sentence.
3. Each stop shows a badge: "Read from code" or "Observed in run".

Flow C: interrupt and ask

1. The user holds Space or the mic button. Narration pauses at once.
2. Speech becomes text, and the answer streams back with citations. The viewer jumps to the cited lines.
3. The tour offers Resume, Go back or Skip ahead, defaulting to Resume at the exact sentence it stopped on.

Flow D: contribute

1. The user pastes a GitHub issue URL or types a goal.
2. A plan card lists the files to read, the files to change, the tests to run and open risks, each linked to code.

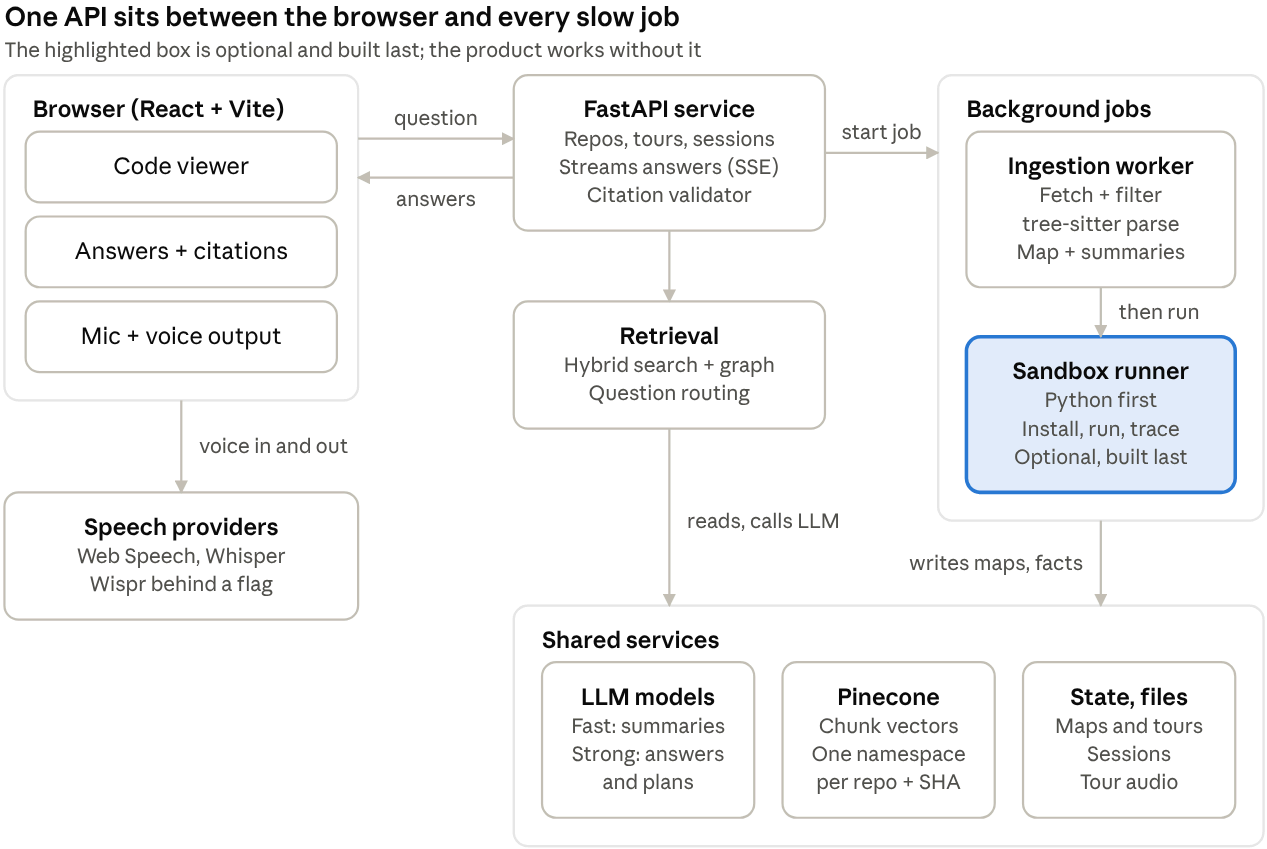
Flow E: return visit

The tool recalls what was already covered for this repo and offers to continue from there instead of starting over.

Failure states are part of the design. If the repo cannot be run, the badge says "static analysis only". If the mic is blocked, the text box works the same way. If retrieval finds nothing, the answer says so instead of guessing.

System architecture

Every question passes through one API, and every slow job runs behind it, so a failed job never blocks an answer.



system architecture · browser, API, background jobs, shared services

Design rules the picture encodes

- Ingestion and the sandbox run as background jobs and report progress through stored events, which keeps the API responsive.
- Everything is keyed by repo URL and commit SHA, so results can be cached and citations stay valid.

- Speech providers are swappable, so the product never depends on Wispr Flow access.
- The sandbox is the only component that runs untrusted code, and it is isolated from the app server.

Ingestion and repo mapping

The goal is to turn a URL into a compact, trustworthy map of the repo in about 30 seconds, before any deep analysis. Every later step starts from this map.

1. **Validate and pin.** Resolve the default branch and the latest commit SHA. All analysis and citations are pinned to that SHA, so line numbers stay valid.
2. **Fetch.** Use a shallow clone (`--depth 1`) under a size cap, for example 200 MB or 5,000 source files. Above the cap, use a sparse checkout of the top-level folders plus the README and tell the user the analysis is partial. Public repos only in version 1.
3. **Filter.** Drop lockfiles, vendored dependencies (`node_modules`, `vendor`), build output (`dist`, `build`), binaries, images, minified or generated files and anything over 1 MB. Respect `.gitignore`.
4. **Detect.** Count languages by extension. Read manifests (`package.json`, `requirements.txt`, `pyproject.toml`, `go.mod`, `Cargo.toml`) to find the stack and frameworks. Find entry points from `main` fields, npm scripts, `if __name__ == "__main__"`, Dockerfile `CMD` and Procfile. Read CI workflow files, which also reveal the real build and test commands.
5. **Parse.** Use tree-sitter to extract functions, classes, imports and exports with line ranges, starting with Python and JavaScript/TypeScript. Build an import graph: which file imports which.
6. **Rank.** Score each file by how many files import it, how close it is to an entry point, and its size. The top of this ranking becomes the tour.
7. **Summarize hierarchically.** Write 1 to 3 sentences per important file, then summarize each folder from its file summaries, then the repo from the folder summaries. Skip tests, config and tiny files at first to control cost.
8. **Emit the repo map.** One JSON object: tree, languages, entry points, run and test commands, import graph, ranked files and summaries, stored against the SHA.

Why a map first: the first answer can start from architecture instead of a file-by-file dump, and the model never needs the whole repo in context. Cache the map per repo and SHA so popular repos are processed once.

Indexing and retrieval

This layer decides whether answers are correct. It is built so a claim without evidence cannot reach the user.

Chunking. Split code by symbol (function or class), not by fixed windows. Each chunk carries a header with path, symbol name and signature, and stays under roughly 400 tokens; long functions split at block boundaries. Docs and READMEs split by heading.

Embeddings and storage. Use one embedding model throughout. Store vectors in Pinecone with one namespace per repo and SHA. Metadata on every chunk: path, start line, end line, symbol, kind (code, doc or test), language. Delete namespaces after a retention window.

Hybrid retrieval.

1. Vector search returns the top 20 chunks.
2. Exact symbol and keyword search (ripgrep-style) catches names that embeddings blur.
3. Graph expansion adds the summaries of files directly imported by, or importing, the top hits.
4. A cheap rerank step cuts the set to the best 8.

Question routing. Classify each question first, because different questions need different evidence:

- Where is X: symbol and keyword search first.
- How does X work: retrieval plus the import graph and any observed call paths.
- What is the architecture: repo map summaries, not code chunks.
- How do I run or test it: manifests, CI files and README commands.

Citation enforcement. The model sees only retrieved chunks, each with an ID, and must return answer segments tagged with chunk IDs. A validator then checks that every ID exists and every line range is inside its file. Uncited segments are removed or shown labelled as inference. If retrieval confidence is low, the answer says "I could not find this in the repo" instead of guessing.

Scale note. A mid-size repo of a few hundred source files should produce a few thousand chunks (an estimate to confirm in the first spike), which keeps embedding cost small.

Execution-grounded analysis

This is the moat: run the repo in a sandbox, observe what actually executes, and attach those observations to explanations. Static readers cannot do this, and it is where most hallucinations come from. It is also the riskiest part, so it ships last and never blocks the product.

Scope. Version 1 runs Python repos only, and only those with a test suite or a clear start command. Everything else uses static analysis and is labelled that way. Node.js is the second target once Python is solid.

Sandbox rules.

- Untrusted code never runs on the app server.
- Containerised, non-root, no secrets in the environment, ephemeral.
- Network egress limited to an allowlist of package registries.
- Hard limits on CPU, memory and time, for example 2 CPUs, 2 GB and 120 seconds for install (starting values to tune).
- Hosted sandboxes such as E2B or Modal save days compared with hardening your own Docker and gVisor setup. Check current pricing and free tiers before choosing.

Pipeline.

1. **Derive commands** from CI workflow files and the README. Fall back to ecosystem defaults such as `pip install -r requirements.txt` and `pytest`.
2. **Install** with a time limit. On failure, record the error, mark the repo static-only and show the user the real reason.
3. **Run tests with tracing.** Use `coverage` with per-test contexts to record which functions each test executes. Cap the output size.
4. **Trace one flow** for web apps that have a start command: start the server, call a few discovered routes, and record the call path with a lightweight tracer filtered to repo files.
5. **Store observations** as facts with provenance: the claim, the command, the commit SHA and the time. Example: "`test_login` executes `auth/session.py:create_token`, then `db/users.py:get_user`".
6. **Use them.** Retrieval includes observation facts, and the tour shows the "Observed in run" badge only on claims they support.

Honesty limits. Coverage shows what ran during tests, not why or how often. Word claims accordingly: "executed during the test suite", never "this is always how it works".

Fallback. If anything in this section fails, the product still works in static mode. That is the cut line for the whole feature.

Guided tour engine

A tour is an ordered list of stops. Each stop holds a file, a line range, a title, a narration script of 60 to 90 words (about 25 to 35 seconds spoken), the highlights to show, and its citations.

Generating the tour

1. **Pick stops** from the repo map: what the README claims, the entry point, the top-ranked core modules, the data layer, one test as a usage example, then config and deploy. Cap at 5 to 8 stops.
2. **Order by dependency.** Walk the import graph so a concept is taught before a stop relies on it.
3. **Write each narration** with an LLM given that stop's code, the file summary, any observation facts and short summaries of earlier stops. The last input prevents repetition and gives natural transitions.
4. **Output structured JSON:** title, narration split into sentences, highlight ranges tied to sentence numbers, and citations.
5. **Validate** with the same citation validator as Q&A. Every line range must exist.
6. **Adapt to level.** Beginner, intermediate and expert change vocabulary and depth, not the stops.

Playback and sync. The client plays sentences in order and moves the viewer highlight when each sentence starts. Sentence-level sync is enough; word-level timing is not worth the cost.

Interruption as a state machine

1. Idle: nothing playing.
2. Playing: narrating stop N, sentence K.
3. Paused: the user pressed push-to-talk. Audio stops, and the position (N, K) is saved.
4. Answering: the question is answered and its citations shown in the viewer.
5. Resuming: Resume continues at (N, K). Go back replays stop N. Skip ahead moves to stop N+1.

Covered-topics list. The session records stop IDs and concept tags already covered. Answers can add "show me" stops, and the planner and the personalization layer both read this list.

Length target. 3 to 5 minutes in total and no stop over 40 seconds. A tour that runs longer loses the listener.

Voice layer

Wispr Flow is a dictation product, and it has an enterprise API for voice-to-text. Access is not self-serve as far as the public material shows, so the plan must work without it. The fix is one interface with swappable providers.

Speech input options

Option	How it plugs in	Strength	Risk
Web Speech API	Built into the browser, called from the React client	Free, no server, live partial text	Mostly Chrome-based browsers, accuracy varies, audio goes to the browser vendor
Whisper (hosted API or self-run faster-whisper)	Client records audio, backend transcribes	Good accuracy, works in any browser	Per-minute cost or GPU needs, no live partials without extra work
Wispr Flow API	Provider behind a feature flag	The product named in the brief, strong dictation quality	Access and terms unknown, may need an enterprise agreement

Provider interface. One contract: `start()`, `onPartial(text)`, `onFinal(text)`, `stop()`. Build the Web Speech provider on day one, Whisper second, and Wispr behind a flag if access is granted. Send the access request to Wispr on day one and treat any answer as a bonus.

Zero-integration Wispr path. Wispr Flow types into whatever text field has focus, so a plain question box already works for anyone who runs the desktop app. That is a fair thing to say in the demo, but it is not an integration, so do not claim one.

Push-to-talk, not always-on. Hold Space or the mic button. This avoids the app hearing its own voice, saves battery and privacy, and gives a clear moment to pause narration.

Speech output

- Browser speech synthesis is free and instant but sounds robotic. Use it as the fallback.
- A cloud TTS service sounds natural but adds cost and latency.
- **Pre-render the tour audio.** Tours are deterministic, so generate each stop's audio once while the user reads the overview, store it, and replay it free on later visits.
- **Stream answers sentence by sentence.** Split the LLM stream at sentence ends, send each sentence to TTS, queue the audio, and start playing after the first sentence. Target under 2 seconds to first audio (a goal to measure).

Interruption. Pressing push-to-talk pauses audio, cancels pending TTS requests and aborts the LLM stream, so nothing speaks over the user.

Speakable text. Narration is written to be heard: "the get user function", not backticks, and line numbers appear on screen but are not read aloud.

Contribution planner

The planner turns understanding into action. Input is a public GitHub issue URL or a free-text goal, and output is a plan card the user can follow.

Steps

1. **Read the issue** through the GitHub API: title, body, labels and comments.
2. **Extract intent and keywords**, then run hybrid retrieval with import-graph expansion to find candidate files.
3. **Choose the minimal edit set:** files to change, files to read for context, and tests to extend or add. Each choice carries a reason and a citation.
4. **Find the workflow:** the test command from the repo map or observations, plus `CONTRIBUTING.md`, the PR template and the detected formatter or linter.
5. **Assemble the plan card:** a root-cause hypothesis labelled as a hypothesis, ordered steps, files with line ranges, a test plan, open risks, and suggested questions for the maintainers.

What it will not do in version 1

- It will not write the patch or open a pull request. A draft patch is a later phase.

- It will not claim the plan is correct. It shows confidence, and says "I could not locate the relevant code" when retrieval is weak.

Why it matters. The tour explains the repo, but the plan is what people come back for. It is also the clearest answer to "so what?" in a pitch. The success signal is a user saying "I knew where to start".

Personalization

Personalization means the tutor does not repeat itself and speaks at the right level. Version 1 keeps it deliberately small.

Learner profile

- **Level:** beginner, intermediate or expert, chosen with three buttons at the start.
- **Goal:** understand, contribute or review.
- **Known stack (optional):** checkboxes such as React or Python, used to pick analogies.

Session memory

- Stop IDs already heard and concept tags covered.
- Questions asked, and a per-repo glossary of terms already explained.

How it is used

- Skip concepts already covered and link back instead ("as we saw in the auth module").
- Change depth and vocabulary by level.
- A Recap button summarizes what has been covered so far.

Storage and privacy. Keep the profile and progress in Firestore under an anonymous ID, with login later. Store no source code in the profile, and give the user a Reset button.

Cut line. Level, goal and covered stops are in version 1. Known-stack analogies and the glossary are extras that go first if time is short.

Tech stack

The stack reuses tools you already know, so learning time goes into the hard parts: retrieval quality, the tour and the sandbox.

Layer	Choice	Why	Alternative
Frontend	React + Vite	Already in your core stack, fast to iterate	Next.js if you want server rendering
Code viewer	CodeMirror 6, read-only	Light, simple line-highlight decorations	Monaco, heavier but familiar
Backend API	FastAPI (Python)	Async, easy streaming over SSE	Node.js with Express
Background jobs	FastAPI background tasks first	No extra infrastructure	Redis with RQ or Celery once load grows
Code parsing	tree-sitter	Language-aware symbols and imports	Regex fallback for unsupported languages
LLMs	A cheap fast model for bulk summaries, a stronger model for Q&A and planning, behind one wrapper	Cost control, easy to swap providers	A single model if simplicity wins
Embeddings and search	Pinecone, one namespace per repo and SHA	Already used in your projects	pgvector if you want fewer services
Sandbox	Hosted sandbox such as E2B or Modal	Saves days of security work	Docker with gVisor on your own server
State and files	Firestore for profiles, tours and sessions; Storage for audio	Already used in your projects	Postgres plus object storage
Speech	Provider interface: Web Speech, Whisper, Wispr behind a flag	Never blocked on one vendor	Cloud-only speech
Hosting	Static host for the frontend, a container host for the API, sandbox separate	Simple, cheap	One VM for everything

Caching rule. Key the repo map, index, tour and audio by repo URL and SHA. A second visitor to the same repo costs almost nothing, and the demo repos can be pre-warmed.

Check current free tiers and pricing for every hosted service before committing; they change often.

Data model and API

Core entities

- **Repo:** id, URL, pinned SHA, status, size stats.
- **RepoMap:** the JSON map from ingestion, keyed by repo and SHA.
- **Chunk:** id, repo, path, start line, end line, symbol, kind. The vector itself lives in Pinecone.
- **Observation:** id, repo, claim, command, evidence, SHA, time.
- **Tour and Stop:** level, ordered stops, narration sentences, highlights, citations, audio URL.
- **Session:** user ID, repo, tour, current position (stop and sentence), covered topics.
- **Message:** session, role, text, citations.
- **Plan:** repo, issue URL, plan JSON.

Endpoints

Method and path	Purpose	Returns
POST /repos	Submit a repo URL	repo ID and status
GET /repos/{id}/events	Stream progress (SSE)	stage events and the partial overview
GET /repos/{id}/map	Fetch the repo map	RepoMap JSON
POST /repos/{id}/tour	Generate a tour for a level and goal	Tour with stops
GET /tours/{id}/stops/{n}/audio	Pre-rendered narration	audio file
POST /sessions/{id}/ask	Ask a question (SSE)	token, sentence, citation, done events

Method and path	Purpose	Returns
POST /repos/{id}/plan	Plan from an issue URL or text	Plan JSON
POST /speech/transcribe	Server-side transcription when used	text

Stream event types: `stage`, `token`, `sentence`, `citation`, `done`, `error`. The `sentence` event is what feeds text-to-speech, and the `citation` event is what moves the viewer.

Evaluation

The benchmark is what turns a good demo into a credible claim. Plan about 2 to 3 hours to build it, and run it after each major phase.

Benchmark set. Use 8 to 10 public repos of mixed size in two languages, none written by you. Write 5 questions per repo covering where-is, how-does-it-work, how-to-run, architecture and change-planning. That gives about 50 questions, each with a ground-truth answer and the correct files.

Metrics

- **Citation validity:** percent of cited files and line ranges that exist. This is checked automatically and should be 100% because the validator enforces it.
- **Citation support:** percent of claims whose cited code really supports them. Use an LLM judge, with a human spot-check on about 20% of items.
- **Answer correctness:** graded 0, 1 or 2 against ground truth with a written rubric.
- **Retrieval hit rate:** the ground-truth file appears in the top 8 retrieved chunks.
- **Run success rate:** percent of repos where install and tests succeed in the sandbox. Report it honestly even if low.
- **Latency and cost:** time to overview, time to first audio, answer time at the median and 95th percentile, and cost per repo.

Ablation conditions

Condition	What is added	Hypothesis to test
A: Plain RAG	Chunks and vector search only	Baseline

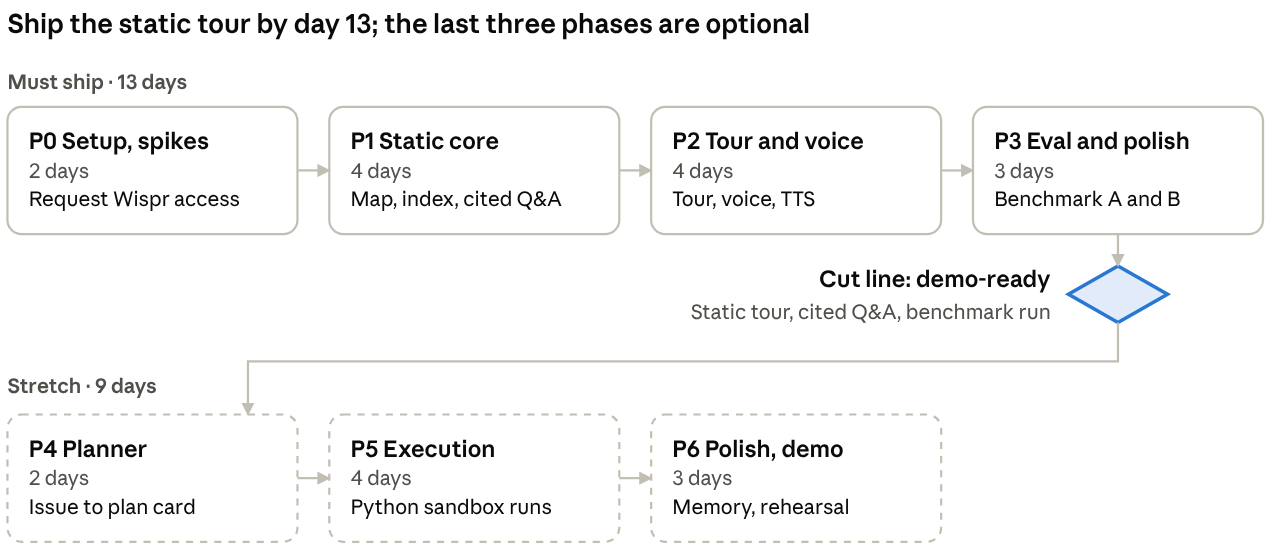
Condition	What is added	Hypothesis to test
B: Map and graph	Repo map, summaries, import-graph expansion	Better on architecture and where-is questions
C: Execution	Observed test and run facts	Better on how-does-it-work questions, with fewer unsupported claims

Fill in the measured numbers only after running them; do not pre-write results.

Small user study. Ask 5 people to find where to add a given feature in an unfamiliar repo, with and without the tool, and record time to the first correct file. Five people is not statistically strong, so present it as a pilot.

Build phases

The static tour, voice and benchmark ship by day 13; the planner, sandbox and polish phases are stretch work that can be dropped.



build roadmap · 7 phases, 1 cut line

The plan assumes one developer and 22 working days. No deadline was given, so treat that as a placeholder and replace it with your real date.

What each phase delivers

- **P0, 2 days:** working spikes for repo fetch, tree-sitter parsing, a Pinecone insert and query, and speech in and out. Send the Wispr access request on day one.

- **P1, 4 days:** ingestion, repo map, index, Q&A with the citation validator, and a basic viewer plus chat screen.
- **P2, 4 days:** tour generation, highlight sync, push-to-talk, sentence-streamed TTS, and interruption with resume.
- **P3, 3 days:** the benchmark with conditions A and B, caching by repo and SHA, error states, and three pre-warmed demo repos. This is the cut line: a demo-ready product exists here.
- **P4, 2 days:** the issue-to-plan flow.
- **P5, 4 days:** the Python sandbox, observed facts, the "Observed in run" badge, and condition C in the benchmark.
- **P6, 3 days:** level and goal memory, the recap button and demo rehearsal.

If time runs short, cut in this order: P6 first, then P5. Keep P4, because it is cheap and answers the "so what" question in a pitch.

Risks and mitigations

The biggest risks are wrong explanations, scope creep and sandbox failures. Each has a fallback that keeps the product alive.

Risk	Likelihood	Impact	Mitigation and fallback
Wrong or invented explanations	High	High	Citation validator, abstain when evidence is weak, benchmark with a judge and spot-checks
Scope creep beyond the time available	High	High	Phases with a cut line; the static tour plus Q&A must ship before anything else
Sandbox install or run fails on many repos	High	Medium	Static fallback with a visible badge, report the run success rate honestly
Running untrusted code is unsafe	Medium	High	Hosted sandbox, egress allowlist, resource limits, never on the app server

Risk	Likelihood	Impact	Mitigation and fallback
Voice feels slow or awkward	Medium	High	Push-to-talk, sentence streaming, pre-rendered tour audio, text box always available
Not differentiated enough from existing tools	Medium	High	Lead the demo with the tour and the "observed" badge, show benchmark numbers, audit competitors before pitching
Wispr Flow API access not granted	Medium	Low	Provider interface with Web Speech and Whisper, so nothing depends on it
Large repos break latency or cost	Medium	Medium	Size caps, partial mode, caching by repo and SHA, per-repo budget cap
Licensing and privacy of code	Low	Medium	Public repos only, store explanations and index data rather than raw code archives, honor deletion on request

The first two rows deserve the most attention. A confident wrong answer in a voice demo does more damage than a missing feature.

Demo and pitch

The demo is a 4-minute story: a stranger's repo, a spoken tour, an interruption, and a plan.

Demo script

1. **0:00 Hook.** "You were assigned an issue in a repo you have never seen. Normally that costs days."
2. **0:15 Paste.** Paste a pre-warmed repo URL, then show a fresh small repo processing live so the stages are visible.
3. **0:45 Tour.** Start the tour. Point out the highlight moving with the voice and the "Observed in run" badge on one flow.
4. **1:45 Interrupt.** Hold Space and ask "wait, where does authentication happen?" Show the cited answer, the viewer jump, and Resume.

5. **2:30 Plan.** Paste an issue URL and show the plan card with files, tests and risks.
6. **3:15 Proof.** Show the benchmark slide with the three conditions side by side.
7. **3:45 Close.** Next steps: private repos, draft patches, an IDE extension.

Demo safety rules

- Pre-warm three repos and test each end to end at least three times.
- Keep a recorded video of the full flow as a backup.
- Keep the text box ready in case the microphone or venue noise fails.
- Never demo a repo you have not already run through the whole pipeline.

The story for judges. Problem (onboarding to unfamiliar code is slow), difference (observed, spoken, actionable), proof (the benchmark), impact (time to first correct change). Repeat the one-line pitch at the start and the end.

Open decisions

Seven choices are still open. The first two block the schedule, so settle them first.

Decision	Options	Depends on	Default if undecided
Deadline and team size	Not given	Which event or submission this is for	Plan assumes one developer and about 22 working days
Wispr Flow access	Granted, denied, no reply	Reply to your access request	Build Web Speech first and treat Wispr as optional
Execution language scope	Python only, or Python and Node.js	Time left after the tour is stable	Python only
Sandbox	Hosted (E2B, Modal) or self-run Docker	Budget and free-tier limits	Hosted
Repo access	Public only, or GitHub login for private repos	Time and privacy concerns	Public only
LLM mix	One provider, or cheap and strong models mixed	Cost per repo from the first spike	Mixed behind one wrapper

Decision	Options	Depends on	Default if undecided
Name	RepoGuide or something else	Your preference	RepoGuide